

GUÍA BÁSICA PARA PRUEBA TÉCNICA

CRUD de Productos con Vue 3 y Vite

Cómo está estructurado, cómo circulan los datos y cómo se comunica con una API REST.

ENFOQUE	CASO PRINCIPAL
Comprensión práctica del código	Módulo de productos y BaseModal.vue

Proyecto analizado localmente • Vue 3.5 • Vite 8 • Bootstrap 5 • Vue Router • Vue I18n

1. El proyecto en 90 segundos

Es una SPA (Single Page Application) administrativa. El navegador carga una sola aplicación Vue; Vue Router cambia la vista visible sin recargar toda la página. Cada módulo presenta una lista, filtros, formularios en modales y operaciones CRUD contra un backend REST.

Mapa mental



Tecnologías y responsabilidad

Tecnología	Qué hace aquí
Vue 3 + Composition API	Componentes reactivos con <code>ref</code> , <code>reactive</code> , <code>computed</code> , <code>watch</code> y ciclo de vida.
Vite	Servidor de desarrollo, variables <code>VITE_*</code> , alias <code>@</code> y compilación de producción.
Vue Router	Relaciona URL con una vista y la muestra dentro de <code><RouterView /></code> .
Vue I18n	Textos de interfaz en español/inglés; el idioma también viaja a la API mediante <code>?lang=</code> .
Bootstrap + Icons	Diseño, rejilla, tablas, formularios, botones, modales e iconos.
Fetch API	Solicitudes HTTP al backend; no se usa Axios.

Idea clave

La vista no debería conocer detalles repetidos de HTTP. Por eso importa la capa `services/`: traduce acciones del negocio como “listar productos” a una URL y un método HTTP.

2. Cómo está organizado

```
src/
|-- main.js arranque y plugins
|-- App.vue componente raíz
|-- router/index.js mapa URL -> vista
|-- views/ pantallas por dominio
| |-- productos/ ProductosView + ProductoForm
| |-- categorias/ lista + formulario
| |-- marcas/ lista + formulario
| |-- inventario/ lista + formulario
| `-- ... precios, descuentos, etc.
|-- components/
| |-- layout/ shell, navbar y sidebar
| `-- common/ piezas reutilizables
|-- services/ acceso a endpoints REST
|-- utils/apiResponse.js tratamiento de respuestas/errores
|-- i18n/ mensajes es/en
|-- constants/ catálogos fijos del frontend
`-- assets/css/app.css estilos globales
```

Recorrido de arranque

Paso	Archivo	Qué ocurre
1	index.html	Aporta el elemento DOM #app.
2	src/main.js	Crea Vue, carga Bootstrap/CSS, registra router e i18n y monta la aplicación.
3	src/App.vue	Renderiza AppLayout.
4	AppLayout.vue	Muestra sidebar, navbar y el contenido dinámico con RouterView.
5	router/index.js	Elige la vista según la URL, por ejemplo /productos.

Patrón de cada módulo

- ***View.vue**: conserva el estado de la pantalla, carga datos, filtra, abre modales y llama servicios.
- ***Form.vue**: recibe datos por props, valida campos, construye el payload y emite submit.
- ***Service.js**: define endpoints y devuelve datos del backend.
- **Componentes common**: resuelven UI repetida sin conocer el negocio.

Regla para ubicar código

Si cambia la apariencia reutilizable, busca en components/common. Si cambia el flujo de una pantalla, busca en views. Si cambia la URL o el contrato HTTP, busca en services.

3. La comunicación con la API

La URL base viene de `import.meta.env.VITE_API_URL`. El README propone este archivo en la raíz:

```
VITE_API_URL=http://localhost:8080/api
```

Ejemplo real: listar productos

Capa	Código / resultado
Vista	<code>listarProductos(locale.value)</code>
Servicio	<code>api.get('/productos', { params: { lang } })</code>
Cliente HTTP	Construye <code>GET {baseUrl}/productos?lang=es</code> .
Backend	Responde JSON con la lista.
Vista	Asigna el resultado a <code>productos.value</code> ; Vue actualiza la tabla.

Qué hace `src/services/api.js`

- Agrega la URL base y serializa parámetros con `URLSearchParams`.
- Serializa el body como JSON y agrega `Content-Type: application/json` cuando hay datos.
- Cancela después de 10 segundos con `AbortController`.
- Intenta leer JSON; ante estado no exitoso crea un `Error` con `error.response`.
- Expone `get`, `post`, `put` y `patch`. No expone `delete`.

Métodos HTTP usados

Método	Intención	Ejemplo
GET	Consultar	<code>/productos?lang=es</code>
POST	Crear	<code>/productos</code>
PUT	Reemplazar/actualizar	<code>/productos/{id}</code>
PATCH	Cambio parcial	<code>/productos/{id}/estado</code>
DELETE	Eliminar	<code>/inventario/{id}</code> (fetch directo)

Detalle importante

El frontend espera que `VITE_API_URL` exista para los módulos que usan `api.js`. Otros servicios incluyen un fallback a `http://localhost:8080/api`. Esa diferencia puede producir comportamientos distintos si falta el archivo `.env`.

4. Caso completo: pantalla de productos

ProductosView.vue funciona como componente coordinador (o container). Mantiene el estado y conecta UI, formulario y servicios.

Al entrar a /productos

Momento	Acción
onMounted(load)	Ejecuta la carga inicial.
load()	Activa cargando y limpia el error.
Promise.all	Solicita productos, categorías y marcas en paralelo.
Éxito	Actualiza tres refs; el template renderiza la tabla y los selects.
Error	Guarda un mensaje en serviceError.
finally	Desactiva el indicador de carga.

Estado reactivo esencial

Estado	Propósito
productos, categorías, marcas	Datos obtenidos del backend.
cargando / guardando	Evitan UI inconsistente y operaciones duplicadas.
search, status, category	Criterios locales de filtrado.
formOpen / confirmOpen	Controlan los dos modales.
editing / selected	Producto editado o pendiente de cambio de estado.
notice / serviceError	Feedback de éxito y error.

Filtrado

`filtered` es un `computed`: deriva una lista a partir de productos y filtros. No modifica el arreglo original ni consulta nuevamente el backend. Al cambiar una dependencia, Vue recalcula el resultado.

Cambio de idioma

`watch(locale, load)` vuelve a consultar la API cuando cambia el idioma. Esto es distinto de traducir la interfaz: Vue i18n traduce botones y etiquetas; el parámetro `lang` pide al backend nombres y descripciones localizados.

Modelo mental

ref/reactive guarda estado; **computed** deriva estado; **watch** ejecuta un efecto cuando algo cambia; **onMounted** ejecuta un efecto al entrar la vista.

5. Crear y editar un producto

Flujo de creación

Paso	Lugar	Qué sucede
1	Clic en Nuevo	<code>create()</code> limpia <code>editing</code> y abre el modal.
2	Formulario	<code>ProductoForm</code> recibe catálogos y listas de valores existentes.
3	Validación	Normaliza código/SKU, valida obligatorios y duplicados.
4	Evento	Emite <code>submit</code> con un payload listo para la API.
5	Vista	<code>save(data)</code> llama <code>crearProducto</code> .
6	Después	Cierra el modal, recarga datos y muestra aviso de éxito.

Payload aproximado

```
{
  codigo: "PROD-001",
  sku: "SKU-001",
  codigoBarras: null,
  categoriaId: 3,
  marcaId: 2,
  activo: true,
  traducciones: [
    { idiomaId: 1, nombre: "Café", descripcion: "..."},
    { idiomaId: 2, nombre: "Coffee", descripcion: "..."}
  ]
}
```

Flujo de edición

- La lista muestra una traducción a la vez, así que `edit(item)` consulta el producto en español y en inglés en paralelo.
- La vista reconstruye un objeto `editing` con `traducciones.es` y `traducciones.en`.
- `ProductoForm` observa el `prop` con `watch` y copia los datos a su estado interno.
- Al guardar, la vista detecta `Boolean(editing.value)` y usa `PUT` en lugar de `POST`.

Props hacia abajo, eventos hacia arriba

La vista entrega datos al formulario mediante **props**. El formulario no llama directamente a la API: comunica el resultado con `emit('submit', payload)`. Este flujo unidireccional hace más fácil reutilizar, probar y razonar sobre el componente.

6. BaseModal.vue, explicado

BaseModal es infraestructura visual reutilizable. No sabe qué es un producto ni cómo guardar: solo muestra contenido, título, pie y comunica el cierre.

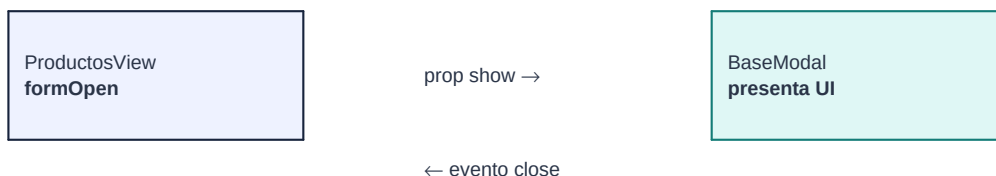
Contrato del componente

Entrada / salida	Significado
prop show	Booleano que decide si el modal existe en el DOM.
prop title	Texto del encabezado.
prop size	Genera una clase Bootstrap como modal-lg.
prop closeOnBackdrop	Permite o impide cerrar al hacer clic fuera del contenido.
evento close	Solicita al padre cerrar; el padre cambia su propio estado.
slot default	Contenido principal, por ejemplo ProductoForm.
slot footer	Botones opcionales del pie.

Las piezas Vue importantes

- `<Teleport to='body'>` mueve el modal al body, evitando problemas de capas, overflow y stacking context del componente padre.
- `v-if='show'` crea y destruye el modal según el estado.
- `@mousedown.self` solo reacciona cuando el clic ocurre exactamente en el fondo, no dentro de `modal-content`.
- `$slots.footer` evita dibujar un pie vacío.
- El `watch` agrega/quita `modal-open` en el body; `onBeforeUnmount` garantiza limpieza.

Quién controla a quién



Ejemplo de uso

```
<BaseModal :show='formOpen' :title='editing ? editTitle : newTitle'
  @close='formOpen = false'>
  <ProductoForm @submit='save' />
  <template #footer>...botones...</template>
</BaseModal>
```

7. Manejo de errores y estados

La vista envuelve operaciones asíncronas con `try/catch/finally`. Este patrón mantiene sincronizados datos, mensajes y spinners.

Bloque	Responsabilidad
<code>try</code>	Ejecuta la solicitud y aplica el resultado.
<code>catch</code>	Convierte el fallo en un mensaje visible para el usuario.
<code>finally</code>	Restablece cargando o guardando, tanto si hubo éxito como error.

Capas actuales de error

- `api.js` produce un error con mensaje HTTP y adjunta `error.response`.
- `apiResponse.js` ofrece funciones para extraer datos, tratar 204 y registrar `messageLog`.
- Algunos servicios duplican su propia función `procesarRespuesta`.
- Productos usa principalmente `error.message` para presentar el error.

Riesgos técnicos que puedes mencionar

Hallazgo	Impacto	Mejora razonable
Dos estilos HTTP	Código duplicado y manejo de errores desigual.	Migrar todos los servicios a un único cliente.
Fallback desigual de URL	Un módulo puede funcionar sin <code>.env</code> y otro no.	Validar la variable al iniciar y usar una política única.
<code>api.js</code> sin DELETE	Inventario no puede usar el cliente común.	Agregar <code>delete(path, config)</code> .
Sin autenticación	No hay token ni manejo de 401/403.	Agregar headers/interceptor equivalente si la prueba lo exige.
Sin paginación real	La UI muestra controles, pero filtra la lista completa localmente.	Definir parámetros <code>page/size</code> y metadatos del backend.

Cómo explicarlo sin criticar de más

“La separación por servicios es correcta. Veo una migración parcial hacia un cliente HTTP compartido; como siguiente paso unificaría los servicios para centralizar timeout, errores, headers y autenticación.”

8. Cómo ejecutar y depurar

Preparación

```
npm install
# crear .env con VITE_API_URL=http://localhost:8080/api
npm run dev
```

El backend debe estar activo. Vite suele mostrar una URL local como `http://localhost:5173`.

Checklist si la tabla no carga

- Revisar la consola del navegador y la pestaña Network.
- Confirmar que `VITE_API_URL` tiene `/api` y reiniciar Vite después de editar `.env`.
- Abrir la solicitud: URL, método, query lang, status y respuesta JSON.
- Comprobar CORS en el backend si el navegador bloquea la solicitud.
- Comprobar que los nombres del JSON coinciden con los usados en la vista: `id`, `codigo`, `nombre`, `activo`, etc.
- Distinguir timeout (10 s en `api.js`), error HTTP y error de red.

Cómo seguir una operación en el código

Pregunta	Dónde mirar
¿Qué dispara la acción?	Evento <code>@click</code> o <code>@submit</code> en la vista/formulario.
¿Qué función corre?	Función del <code><script setup></code> , por ejemplo <code>save</code> .
¿Qué endpoint llama?	Import correspondiente en <code>src/services</code> .
¿Qué se envía?	Payload construido justo antes de <code>emit('submit')</code> .
¿Qué cambia en pantalla?	Asignaciones a <code>refs/reactive</code> y condiciones <code>v-if/v-for</code> .

Prueba rápida de producción

Ejecuta `npm run build`. Si compila, valida sintaxis e integración del bundle; no sustituye pruebas funcionales contra el backend.

9. Preguntas típicas de prueba técnica

¿Por qué separar View y Form?

La vista coordina datos/API; el formulario gestiona entrada, validación y payload. Reduce responsabilidades y permite reutilizar el formulario para crear y editar.

¿Qué diferencia hay entre ref y reactive?

ref envuelve un valor y se accede con `.value` en JavaScript; reactive crea un proxy para un objeto. En el template Vue los desenvuelve automáticamente.

¿Por qué filtered es computed?

Es información derivada. Vue la recalcula cuando cambian productos o filtros y puede reutilizar el valor mientras no cambien dependencias.

¿Por qué Promise.all?

Productos, categorías y marcas son solicitudes independientes. Ejecutarlas en paralelo reduce el tiempo total de carga; si una falla, el conjunto rechaza.

¿Para qué sirve Teleport?

Renderiza el modal bajo body aunque su declaración esté dentro de la vista, evitando límites y conflictos visuales del árbol padre.

¿Cómo viaja el formulario al backend?

ProductoForm emite un payload; ProductosView recibe el evento; productoService elige POST o PUT; api.js serializa JSON y usa fetch.

¿Qué ocurre al cambiar el idioma?

Vue I18n cambia textos locales y el watch de la vista recarga datos del backend con el nuevo parámetro lang.

¿Cómo evitar doble envío?

La vista comprueba guardando, lo activa antes del await y deshabilita controles hasta finally.

¿Qué mejorarías primero?

Unificar todos los servicios en api.js, agregar DELETE, validar VITE_API_URL y estandarizar errores. Después añadir tests y paginación real.

10. Guion corto para presentar el proyecto

Puedes explicarlo así en 60-90 segundos:

“Es una SPA administrativa hecha con Vue 3 y Vite. main.js registra Router e i18n; AppLayout mantiene la navegación y RouterView muestra cada módulo. Los módulos están separados por dominio: una View coordina el estado y las llamadas, y un Form recibe props, valida y emite el payload. La comunicación REST vive en services: para productos se usa un cliente común basado en fetch que toma VITE_API_URL, agrega query params, serializa JSON, aplica timeout y normaliza errores. En productos, la carga obtiene lista, categorías y marcas con Promise.all; crear usa POST, editar usa PUT y el estado usa PATCH. BaseModal es un componente reutilizable controlado por props y eventos, con Teleport y slots. Como mejora, unificaría los servicios que aún usan fetch directo y agregaría DELETE, autenticación y paginación.”

Ruta de estudio recomendada

Orden	Lee	Objetivo
1	main.js → App.vue → AppLayout.vue	Comprender el arranque.
2	router/index.js	Relacionar URLs con pantallas.
3	ProductosView.vue	Entender estado y coordinación.
4	ProductoForm.vue	Seguir props, validación y emit.
5	productoService.js → api.js	Seguir la solicitud HTTP completa.
6	BaseModal.vue	Comprender reutilización, slots y Teleport.
7	Otro módulo	Comparar patrones e inconsistencias.

Ejercicio práctico

- Abre /productos y, con Network visible, crea un registro de prueba.
- Antes de enviar, predice método, URL, query y body.
- Compara tu predicción con la solicitud real.
- Edita el registro y explica por qué se hacen dos GET antes del PUT.
- Cambia el idioma y observa qué llamadas se repiten.
- Provoca un error controlado (backend apagado o dato inválido) y sigue el mensaje hasta la UI.

Si puedes narrar esos seis pasos señalando los archivos involucrados, ya entiendes el núcleo del proyecto y puedes defenderlo con seguridad en una prueba técnica.

Documento elaborado a partir del código actual del repositorio. No describe el backend interno; infiere su contrato desde los endpoints y payloads consumidos por el frontend.